iscte TECNOLOGIAS E ARQUITETURA

Software Engineering

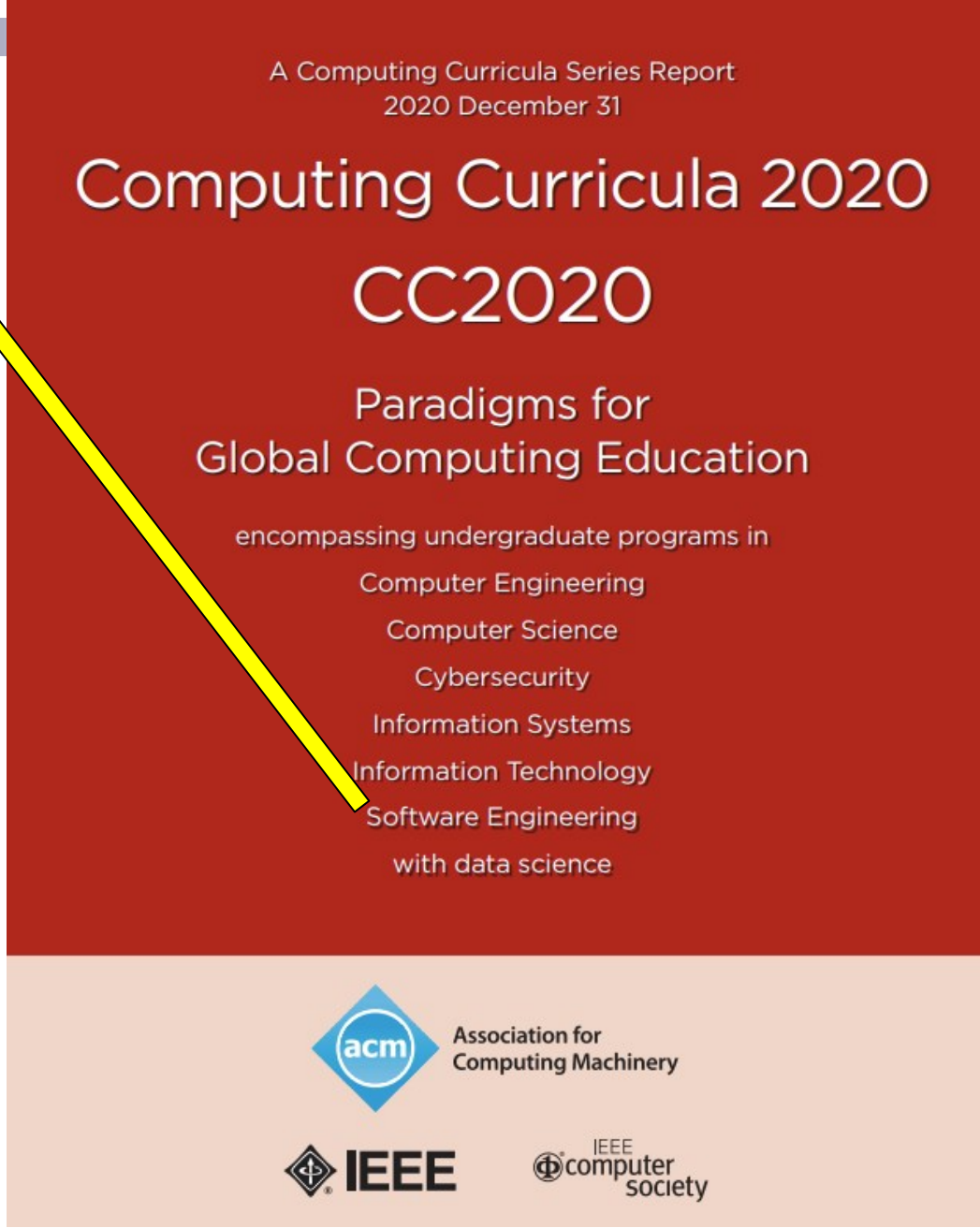
Configuration Management and Version Control with Git

Soft. Eng. Competences

- Software Requirements
- Software Design
- Software Construction
- Software Testing
- Software Sustainment
- Software Process and Life Cycle
- Software Systems Engineering
- Software Quality
- Software Security
- Software Safety
- Software Configuration Manag.
- Software Measurement
- Human-Computer Interaction
- Project Management
- Behavioral Attributes

CDSI

IPM



Software Engineering Body of Knowledge

1. Software Requirements
2. Software Architecture
3. Software Design
4. Software Construction
5. Software Testing
6. Software Engineering Operations
7. Software Maintenance
8. Software Configuration Management
9. Software Engineering Management
10. Software Engineering Models and Methods
11. Software Engineering Process
12. Software Quality
13. Software Security
14. Software Engineering Economics
15. Software Engineering Professional Practice
16. Computing Foundations
17. Mathematical Foundations
18. Engineering Foundations

Refers to the discipline of identifying, organizing, and controlling modifications to software to ensure integrity and traceability throughout its lifecycle. [SWEBOK 4.0, Dec. 2022]

Configuration Management

- Because software changes frequently, systems, can be thought of as a set of versions, each of which has to be maintained and managed.

- Versions implement proposals for change, corrections of faults, and adaptations for different hardware and operating systems.

Configuration Management

- Software Configuration Management (SCM) is concerned with the policies, processes and tools for managing changing software systems.

- You need CM because it is easy to lose track of what changes and component versions have been incorporated into each system version.

What do these systems have in common?



Other civilian areas where a software failure can lead to significant loss of life, injury, or societal damage include:

- Nuclear Power Plants
- Energy Grid Systems
- Fire Safety Systems
- Water Treatment and Distribution
- Chemical and Hazardous Materials Management
- Building and Structural Safety

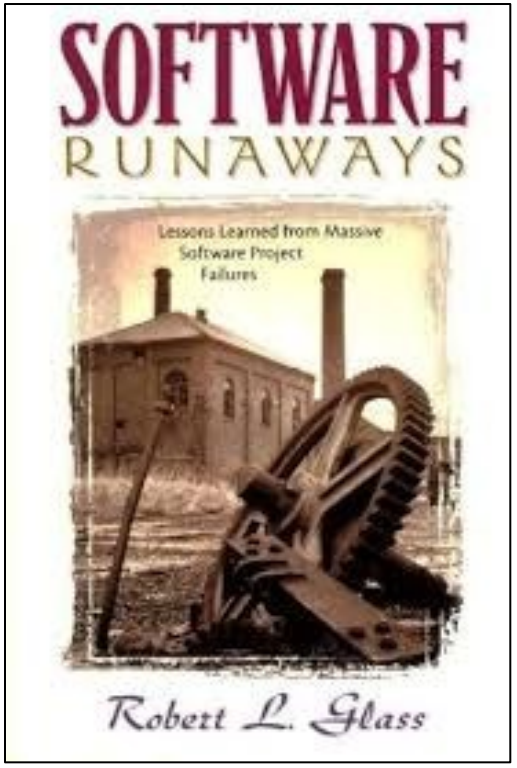
Military (defense) software is mostly critical

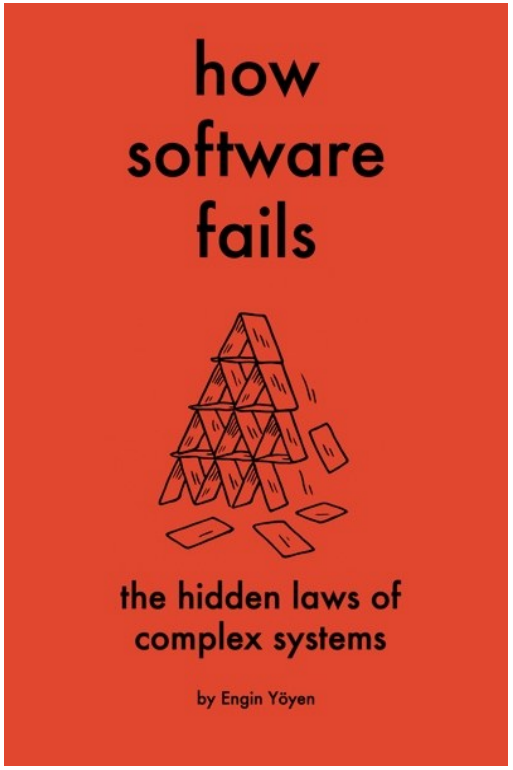


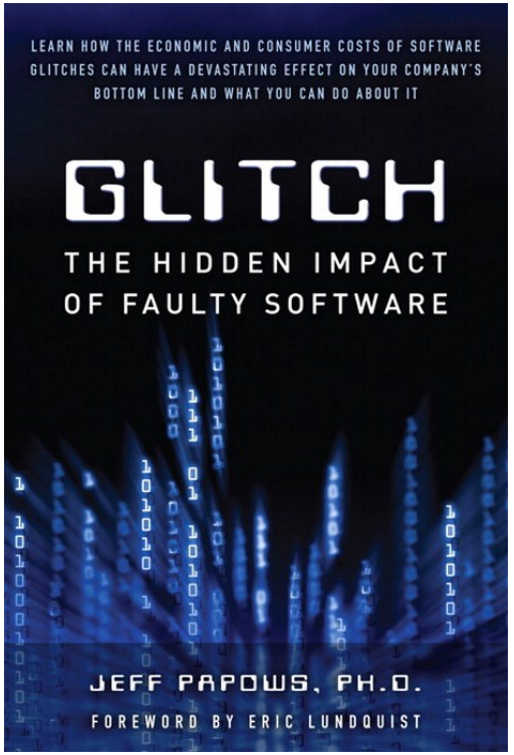
Software failures in military systems can have serious, even catastrophic consequences for civilians, often turning them into unintended, innocent victims:

- Targeting Errors (Friendly Fire or Misfire)
- Collateral Damage from Autonomous Systems
- Cyber Warfare Gone Wrong
- Navigation and Communication Failures
- Unintended Escalation

When critical software fails …







Prevention is better than the cure, but when a failure occurs we need to perform auditing for accountability purposes. A Software Configuration Management (SCM) Plan must be in place.

SCM Processes

- Change control process
- Configuration audit
- Release management
- CM planning

Change Control Process

- Submission of Change Request (CR)
- Technical and business evaluation and impact analysis
- Approval by Change Control Board (CCB)
- Engineering Change Order (ECO) is generated stating
  - changes to be made
  - criteria for reviewing the changed CI
- CI's checked out
- Changes made and reviewed
- CI's checked in

A partially completed change request form (a)

Change Request Form

Project: SICSA/AppProcessing **Number:** 23/02
Change requester: I. Sommerville **Date:** 20/01/09
Requested change: The status of applicants (rejected, accepted, etc.) should be shown visually in the displayed list of applicants.

Change analyzer: R. Looek **Analysis date:** 25/01/09
Components affected: ApplicantListDisplay, StatusUpdater

Associated components: StudentDatabase

A partially completed change request form (b)

Change Request Form

Change assessment: Relatively simple to implement by changing the display color according to status. A table must be added to relate status to colors. No changes to associated components are required.

Change priority: Medium
Change implementation:
Estimated effort: 2 hours
Date to SGA app. team: 28/01/09 **CCB decision date:** 30/01/09
Decision: Accept change. Change to be implemented in Release 1.2
Change implementor: **Date of change:**
Date submitted to QA: **QA decision:**
Date submitted to CM:
Comments:

Configuration Audit

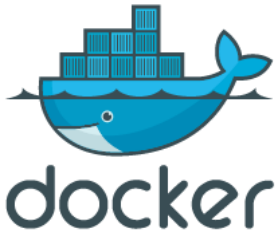
- Independent review or examination to assess if a product or process complies with specification, standards, contractual agreement, or other criteria
- Examples
  - Verifies that CIs are tested to satisfy functional requirements
  - Verifies that baseline contains necessary and correct CI versions
  - Ensures that changes made to a baseline comply with the configuration status reports

Release Management

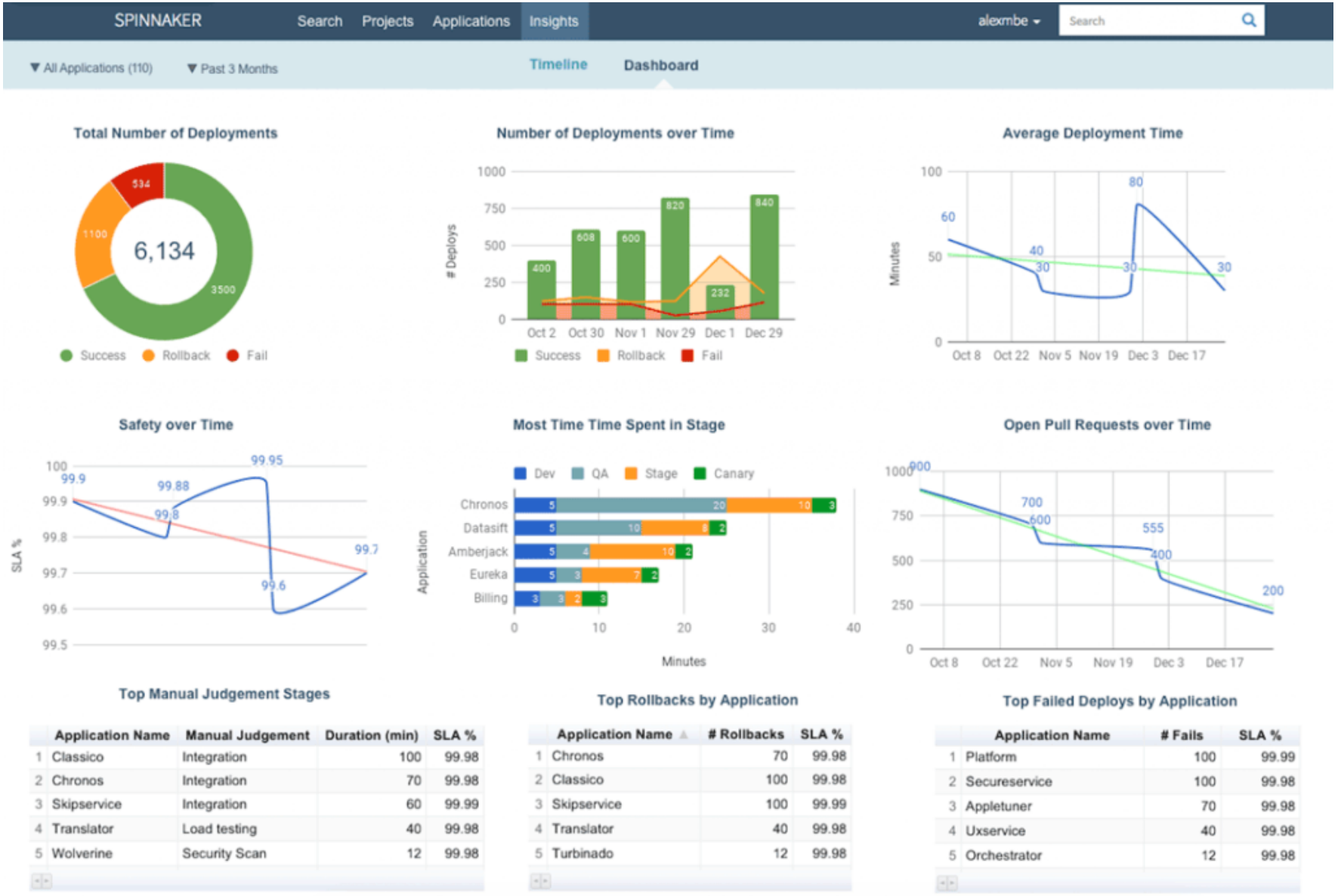
- Creation and availability of a new version of software to the public

- Release content / format
  - Source code + build script + instructions for developers
  - Executables, including libraries (binaries)
  - Libraries (binaries)
  - Data and configuration files
  - Installation scripts
  - User documentation

Note: the release can be packaged in a container



Release Management (tool)



Top Manual Judgement Stages

	Application Name	Manual Judgement	Duration (min)	SLA %
1	Classico	Integration	100	99.98
2	Chronos	Integration	70	99.98
3	Skipservice	Integration	60	99.99
4	Translator	Load testing	40	99.98
5	Wolverine	Security Scan	12	99.98

Top Rollbacks by Application

	Application Name	# Rollbacks	SLA %
1	Chronos	70	99.98
2	Classico	100	99.98
3	Skipservice	100	99.99
4	Translator	40	99.98
5	Turbinado	12	99.98

Top Failed Deploys by Application

	Application Name	# Fails	SLA %
1	Platform	100	99.99
2	Secureservice	100	99.98
3	Appletuner	70	99.98
4	Uxservice	40	99.98
5	Orchestrator	12	99.98

Make a CM Plan



- Standards
  - IEEE Std 828 (SCM Plans)
  - ANSI-IEEE Std 1042 (SCM), …
- CM plan components
  - What will be managed (list and organize CIs)
  - Who will be responsible for what activities (roles and tasks)
  - How to make it happen (design processes for change requests, task dispatching, monitoring, testing, release, etc.)
  - What records to keep (logs, notes, configurations, changes, etc.)
  - What resources and how many (tools, money, manpower, etc.)
  - What metrics to measure progress and success

Version Control

- Required for handling change, because:
  - Avoids (involuntary) overwrite of changes
  - Detects conflicts
  - Records a history of changes
  - Enables efficient teamwork
  - Enables efficient development of multiple variants at the same time

Some Simple Scenarios

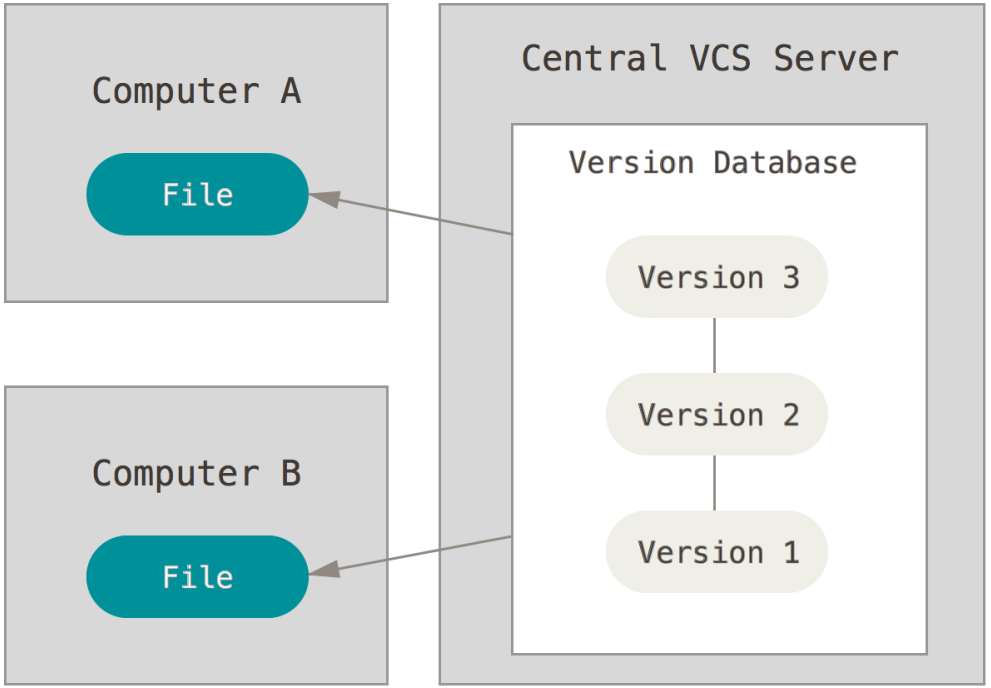
- Developer A wants to see latest version of Foo.java and its change history since last week.
- B needs to revert foo-design.doc to its version two days ago.
- B makes a release of the project and he needs to know what items to include and which version.

Some Simple Scenarios

- A lives in New Dehli, India and B lives in Boston, US, they want to work on HelloWorld.java together.
- In the latest release, a serious bug is found and manager C wants to track what changes caused the bug, who made those changes and when.
- C wants to get reports about current project progress to decide if she needs to hire more programmers and delay the alpha release.

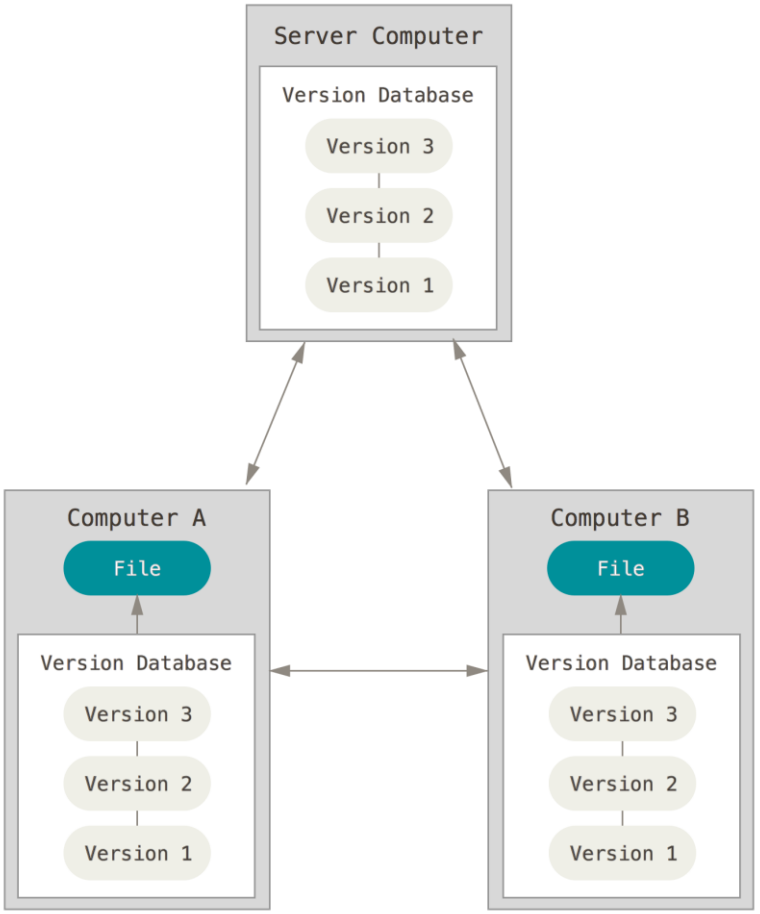
Centralized *vs* Distributed SCM Systems

Centralized Version Control



Centralized *vs* Distributed SCM Systems

Distributed Version Control



SCM Terminology

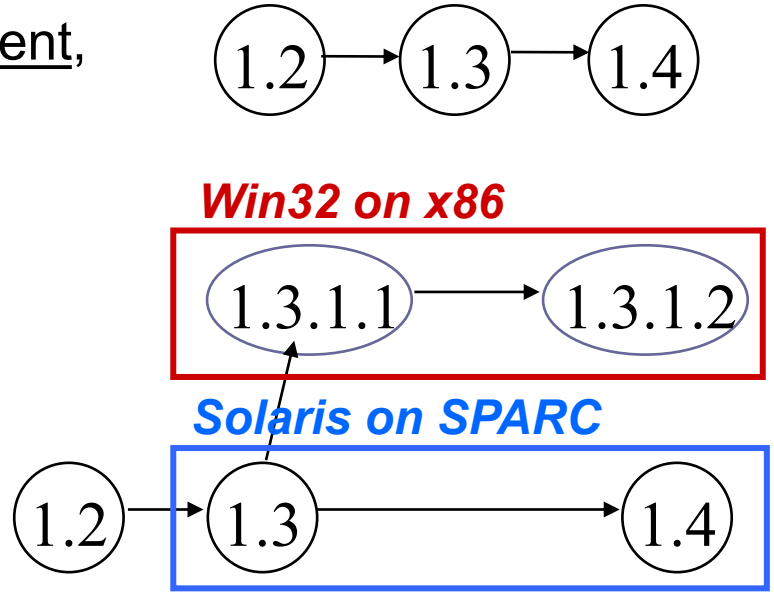
- Configuration Item (CI)
- Revision, Version and Variant
- Configuration
- Baseline
- Workspace

Configuration Item (CI)

- Examples:
  - Management plan
  - Requirement
  - Design specification
  - Source code and executable code
  - Test specification, data, and records
  - Log information
  - User documentation
  - Library and supporting software
  - Bug reports, etc.

Revision, Variant, Version and Branch

- **Revision**: a CI linked to another via revision-of relationship, and ordered in time
- **Variant**: functionally equivalent versions, but designed for different settings, e.g. hardware and software
- **Version**: a CI at one point in its development, includes revision and variant
- **Branch**: a sequence of versions in the timeline



Configuration

- An arrangement of functional CIs according to their nature, version and other characteristics.
- Sometimes, configuration needs to record environment details, e.g. compiler version, library version, hardware platform, etc.
- Simple examples
  - Unix Makefile, Ant buildfile, Maven POM file







Baseline

- A collection of item versions that have been formally reviewed and agreed on, a version of configuration

- Marks milestones and serves as basis for further development

***NOTE**: All releases are baselines, but the opposite is not true, i.e. some baselines are only created as a milestone for internal development.* These are often used within the development team to track progress, ensure stability, and serve as checkpoints before a product reaches a release stage.

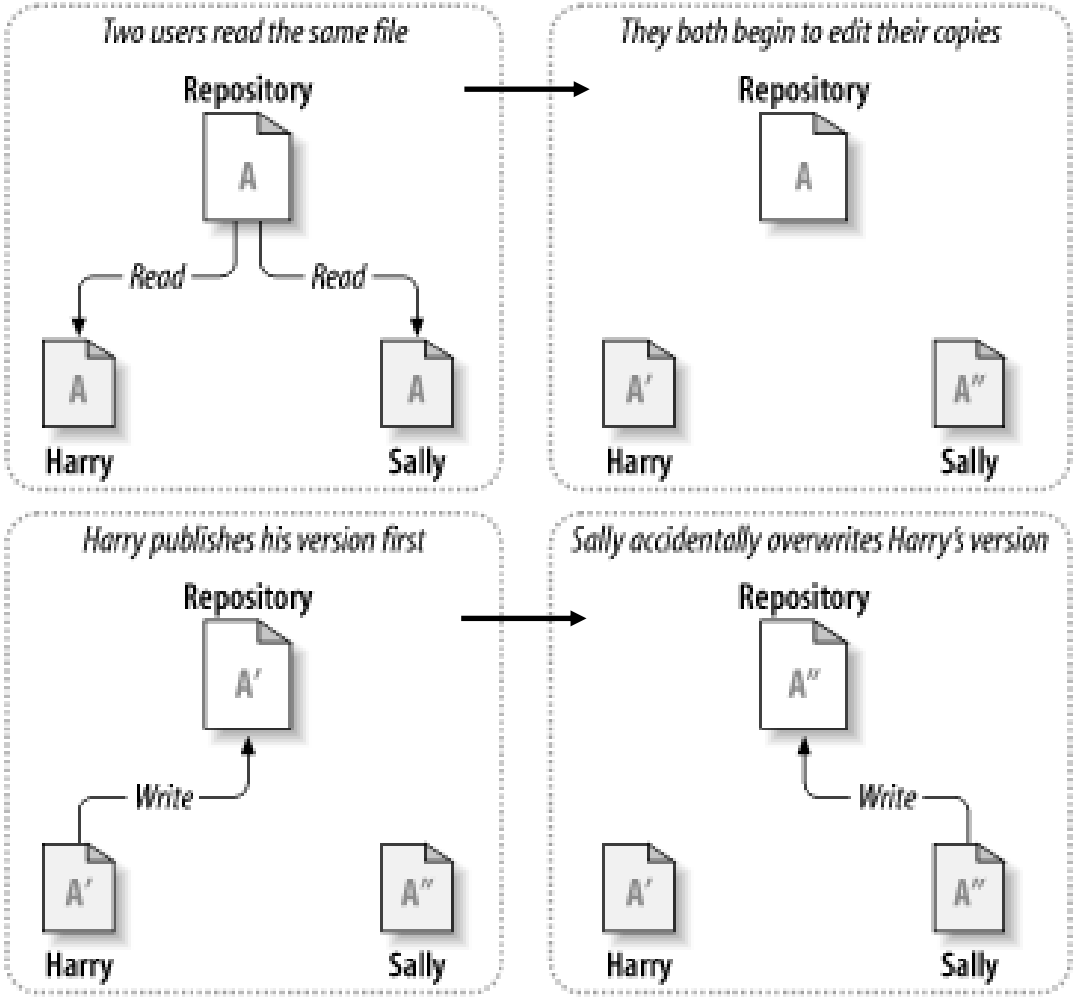
- In critical software systems, a baseline can only be changed via formal change management process

Workspace (aka sandbox)

- An isolated environment where a developer can work (edit, change, compile, test) without interfering with other developers
- The workspace is usually a set of directories associated with your project (src, test, doc, …) in the IDE
- Common Operations
  - Add resources into version control
  - Checkout: get a version into workspace
  - Update: get latest version on the default branch
  - Checkin/commit: commit changes to the repository

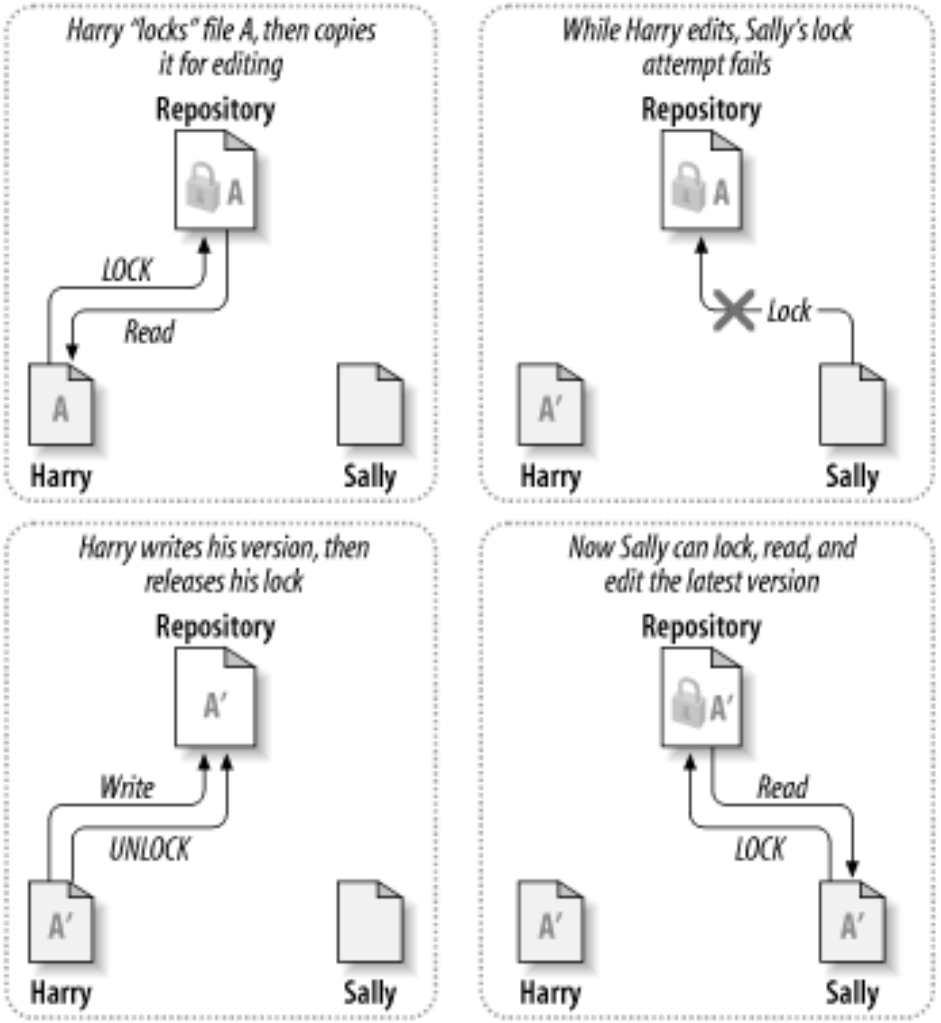
Version Control Models

- Basic problem of collaborative work



Version Control Models

- Model 1-Pessimistic: lock-modify-unlock

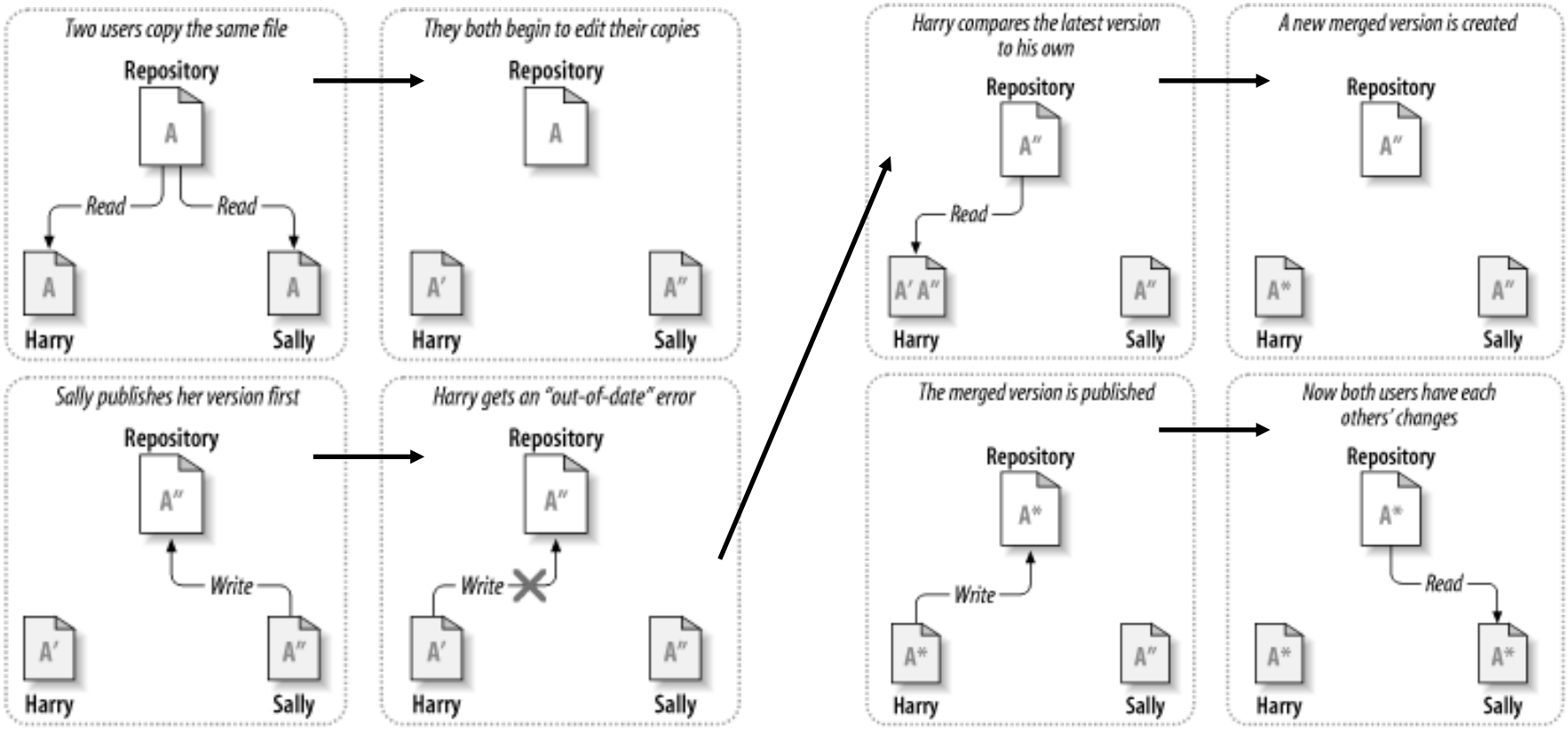


Problems:

- Forget to unlock
- Parallel work not possible
- Deadlock

Version Control Models

Model 2-Optimistic: copy-modify-merge



Version Control Systems



- SCCS (Source Code Control System, 1975)
  - Marc Rochkind, Bell Labs, included in Unix



- RCS (Revision Control System, 1985)
  - Walter F. Tichy, Purdue Univ., managed by Gnu



- CVS (Concurrent Versions System, 1989)
  - Simple, with a central repository



- SVN (Apache Subversion, 2000)
  - Optimized cvs-like version, but still centralized



- Hg - Mercurial (2005)
  - Distributed revision control system



Version Control Systems



- **Git**
  - Distributed revision control system
  - Created by Linus Torvalds in 2005
  - Is currently supported by most online repository platforms



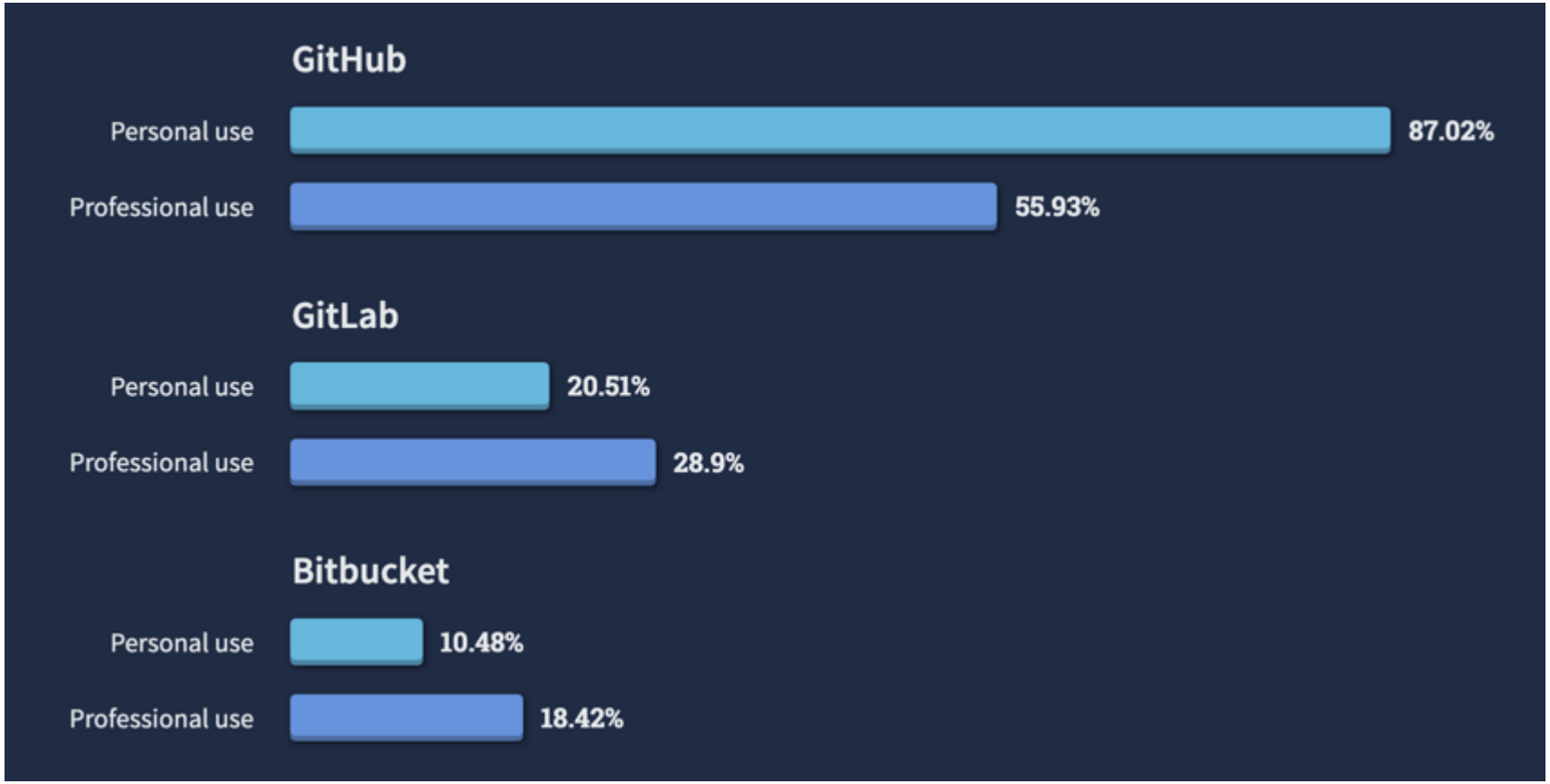








Git-based remote repository platforms



Top 3 version control tools according to the StackOverflow 2022 Developer Survey

Git



- Full history is stored (diachronic traceability)
  - Cryptographic hashes ensure the integrity of the history
- Efficient handling of large projects
  - Only deltas are stored
- Most operations are local
  - More efficient and allowing disconnected work
- Distributed development
  - Multiple developers can work on the same project
- Changes are staged before committed
  - To allow semantic grouping of changed CIs

Git Concepts



- **Working Tree**, directory (subdirectories and files that you edit/add/remove) on your file system associated with a repository (managed as a Git repository).

- **Index (staging area)**, any changes to the Working Tree are noted by the Index (added to the index) and show up as modified files. The contents of the index are the snapshot of the modified files.

- **HEAD**, commit at the top of the current branch. If you 've just checked out the branch (haven't modified files), its content matches the working tree. As soon as you modify anything, it no longer matches.

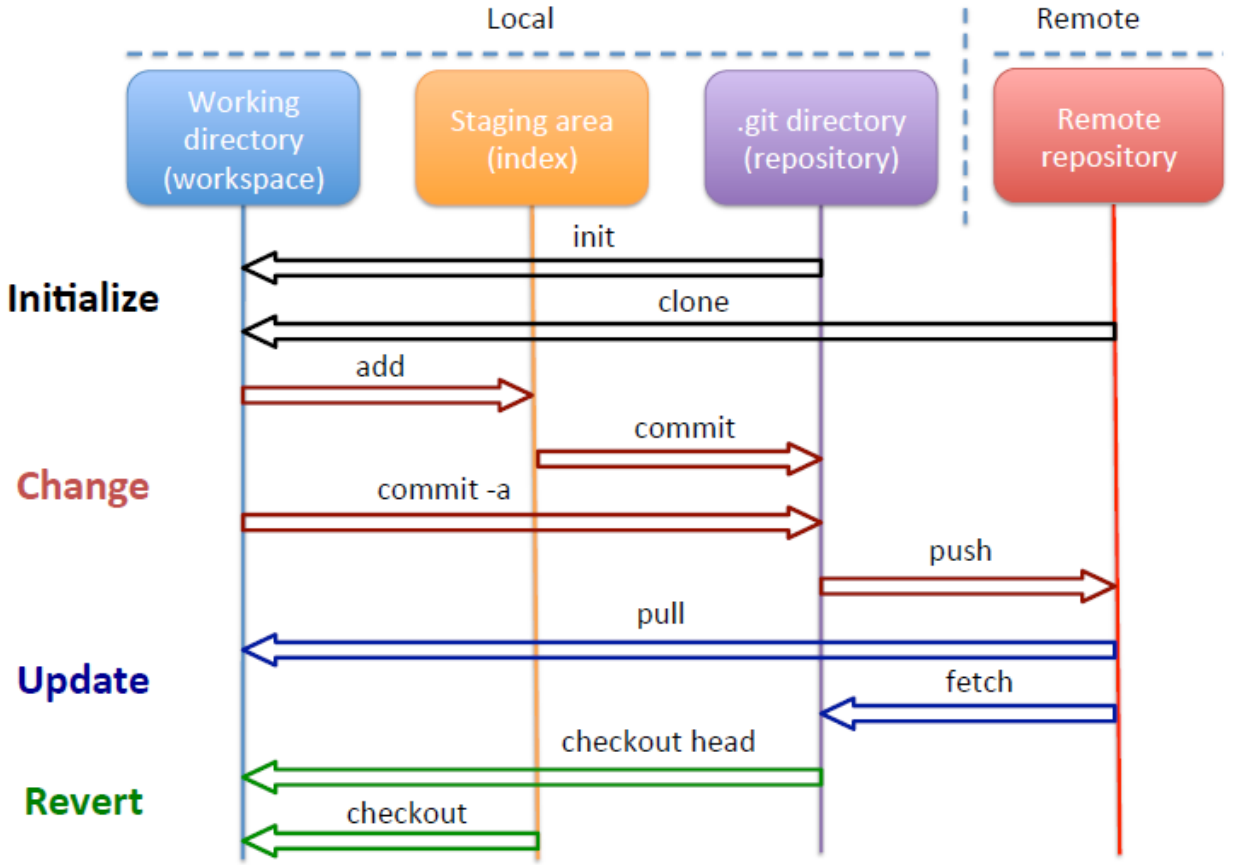
Git Concepts



- A single **git repository** can track an arbitrary **number of branches**, but your **working tree is associated with just one** of them (the "current" or "checked out" branch), and **HEAD** points to that branch.

- **For a given file**, it is possible to have **three separate copies**, the previously commited version (HEAD), the current version on disk (working tree) and a cached version in the index.

Git Concepts



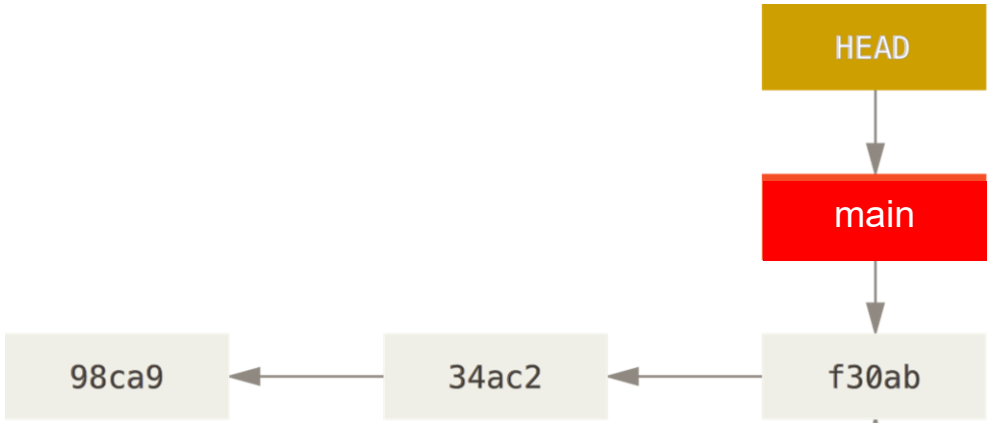
**push*: It is not the content of working directory or staging area that is pushed – it is only the committed changes.

** If you have many changed files and want to commit all, you can skip the 'git add' step using the '-a' (be aware that untracked new files are not included!).

****origin* is the default name for the remote repository.

Git Concepts

- *main* is the default name for the first branch created in the repository.

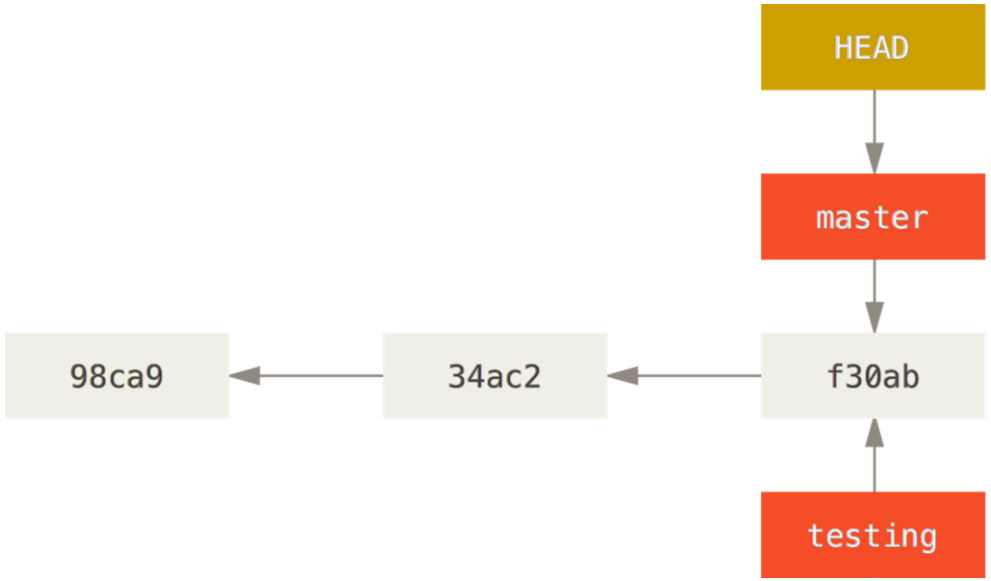


Changed in 2020 from **master** to **main** due to concerns about inclusivity and to remove references to terms with problematic historical connotations.

Git Concepts

- Creating a branch

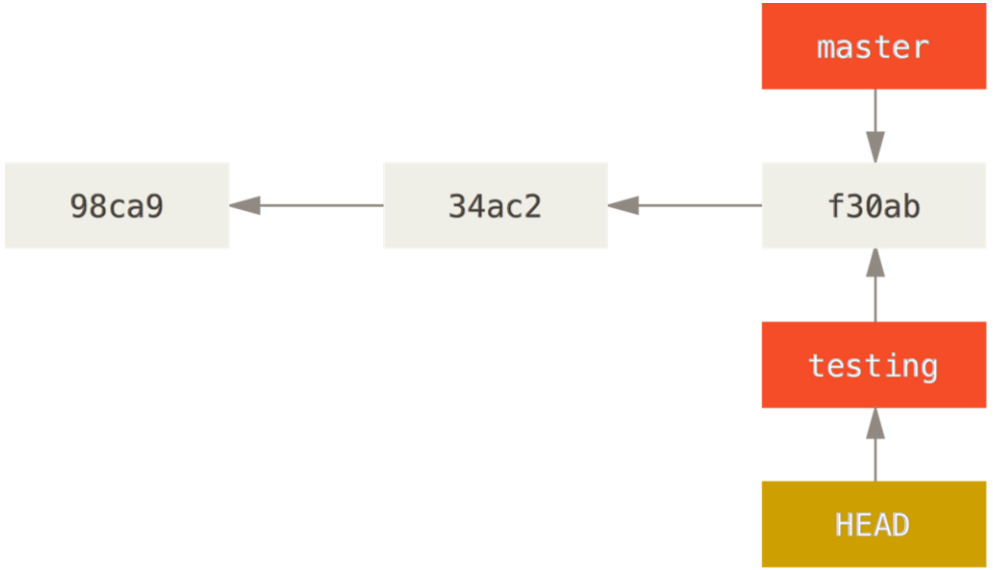
```
$ git branch testing
```



Git Concepts

- Changing to an existing branch

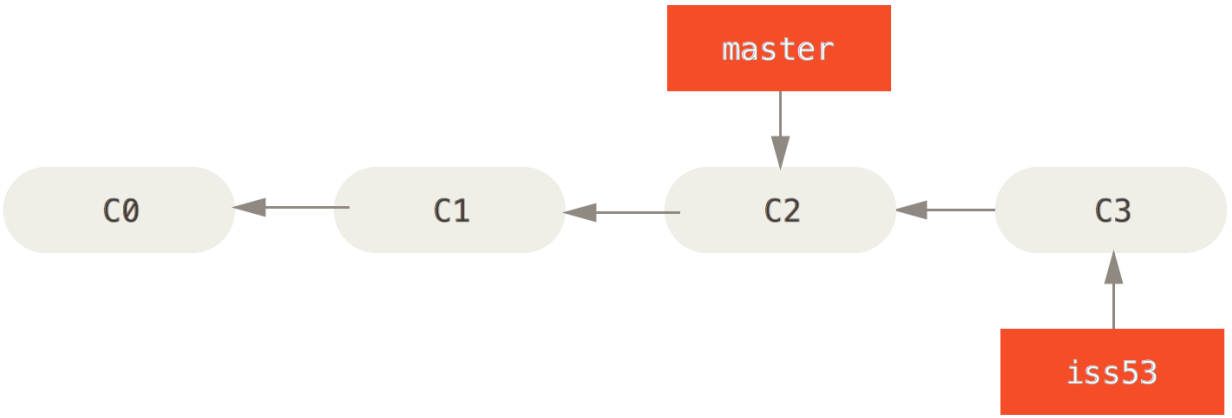
```
$ git checkout testing
```



Git Concepts

- Merging branches

Assume that a new branch has been created to fix an issue (issue #53), the issue has been fixed and committed (C3):

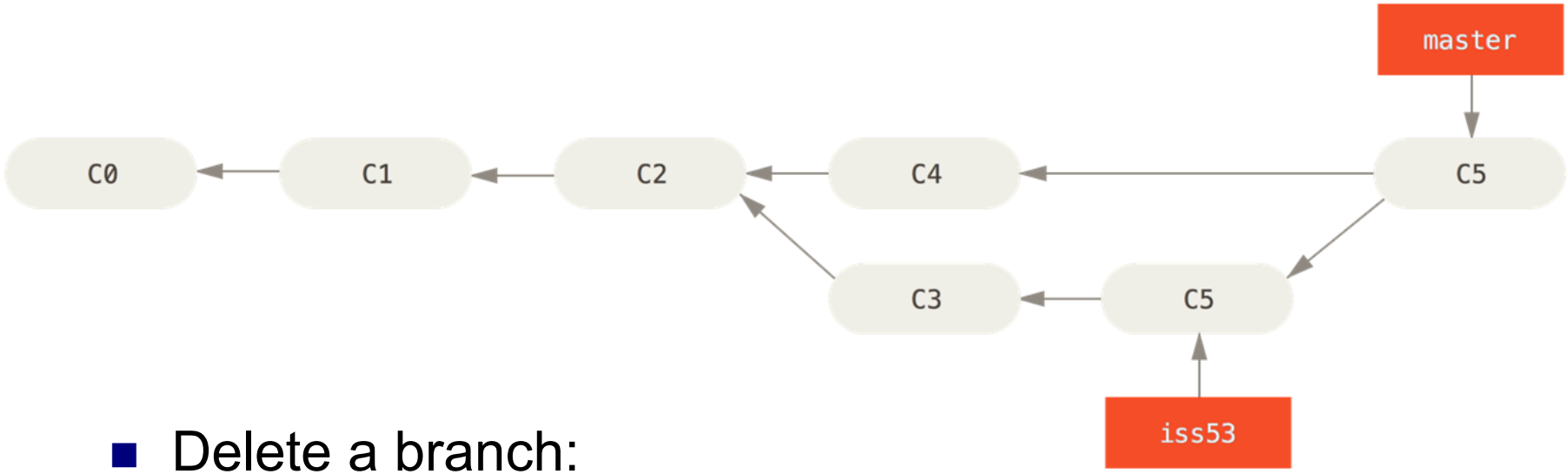


Git Concepts

- Merging branches

  Switch back to the master branch and merge branches:

```
$ git checkout master (Switched to branch ‘master‘)
$ git merge iss53
```



- Delete a branch:

```
$ git branch -d iss53
```

Git Concepts



- **Merge conflicts**

  John and Jane have made incompatible changes to the same file, John has pushed his changes, when Jane has to push her changes, she gets an error:

```
$ git push
! [rejected]        master -> master (fetch first)
…
```

- Fetch the new commits from the remote:

```
$ git fetch
```

- Try to merge:

```
$ git merge
Auto-merging x
CONFLICT (content): Merge conflict in <file>
Automatic merge failed; fix conflicts and then commit the result.
…
```

- Fix the conflicts in the file, commit & push new manually merged version.

Git Concepts



- There are two types of tags:
  - Lightweight tags: pointers to a commit.
  - Annotated tags (recommended type): contains name, email, time & a message.

- Creating an annotated tag:

```
$ git tag –a [tag-name] –m “[message]”
$ git tag –a v1.0 –m “First release”
```

- Showing information for a specific tag:

```
$ git show v1.0
```

- Seeing a list of tags:

```
$ git tag
```

Git Concepts



- By default “git push” does not push tags.
- Pushing a specific tag:

  $ git push origin [tag]

- Pushing all tags:

  $ git push origin --tags

- Tags are automatically cloned and fetched.